

Choosing a Multi-Agent Architecture

The Two Slides Worth Screenshotting, In One Card

The mentor asked the class twice to screenshot a slide during Session 8, and both were about choosing an architecture. This card holds that material in one place, so you can make the decision without rewatching the recording. It also appears inside the Session 8 Session Notes. Start with the prior question: do you need a multi-agent system at all?

Criteria	Single agent	Multi-agent system
Task complexity	The task is straightforward. Multi-agent here is an overkill.	The task is complex and decomposes into smaller tasks.
Speed and latency	You want fast execution. One LLM, simple tools, no delegation overhead.	Speed is not the priority, and a single agent would hit its context limit.
Cost	Cost matters more than squeezing out extra performance.	Accuracy matters more than cost.
Where to start	Always start here.	Only after you have named the concrete limitations of the single agent version.

The rule that overrides everything else

Always start with a single agent system. Identify its limitations. Then decide whether you need to pivot to multi-agent. Do not start with the most complex architecture and try to decompose it into something simpler later. And remember that every hand-off from a coordinator to a sub-agent costs tokens and time, so multi-agent systems buy you accuracy, not speed.

The three patterns, compared

Dimension	Sequential	Supervisor (coordinator)	Hierarchical
Shape	Agent A to Agent B to Agent C, one after another.	A boss agent delegating to sub-agents that never talk to each other.	Coordinators of coordinators, multiple layers deep.

Dimension	Sequential	Supervisor (coordinator)	Hierarchical
Use when	The steps are always in the same order and nothing is decided at run time.	The right next step depends on the task and may loop or skip.	You have multiple distinct domains, for example sub-teams of a digital company.
Number of agents	Few, in a fixed chain.	Few and fixed, typically 3 to 6 workers plus the supervisor.	Many, across several layers.
Predictability	Highest. Least probabilistic.	Lower than sequential, still workable.	Lowest, because of context poisoning travelling through the hierarchy.
Cost and latency	Most budget friendly.	More expensive, acceptable when you need accuracy on a complex task.	Most expensive, and runs the longest.
Debuggability	Easiest.	Harder. You have to track conversations to find the point of failure.	Hardest. Points of failure grow exponentially.
Parallelism	None inside one pipeline. Each agent waits on the previous output.	Possible, but only where one agent's output does not depend on another's.	Possible, and entirely your design choice.
Engineering effort	Low.	Moderate.	High. Needs thorough context engineering to be safe.
Worked example	Research agent drafts, review agent reviews, finaliser agent finalises.	Supervisor delegates to a researcher agent, assesses the result, delegates again.	Supervisor over a research agent that itself commands Google, Money Control and YFinance agents.
In the wild	Code review, email generation, invoice processing, trip and ticket booking.	Customer support routing to billing, technical support or orders. The Session 8 GitHub agent.	Claude Code, OpenClaw, CrewAI by default, marketing and content agent teams.

Why hierarchical is the least predictable: Chinese whispers

One person whispers a word around a circle, and by the time it comes back the meaning has changed. In a hierarchical system, if a small amount of context poisoning happens mid execution, it can change the whole direction of the run and even change the goal. The sub-agents never talk to each other, so the corruption travels through the manager: a poisoned response lives on in the manager's history and trickles down into every sub-agent it delegates to afterwards.

How to use this card: read it top to bottom. Decide single or multi-agent on the four criteria first. If multi-agent, take the cheapest pattern that fits the shape of your problem, and only move down the table when the evaluation tells you to. There is no problem where any of these three cannot be used. The only question is which is most suitable, and you answer it by comparison, experimentation and evaluation.